

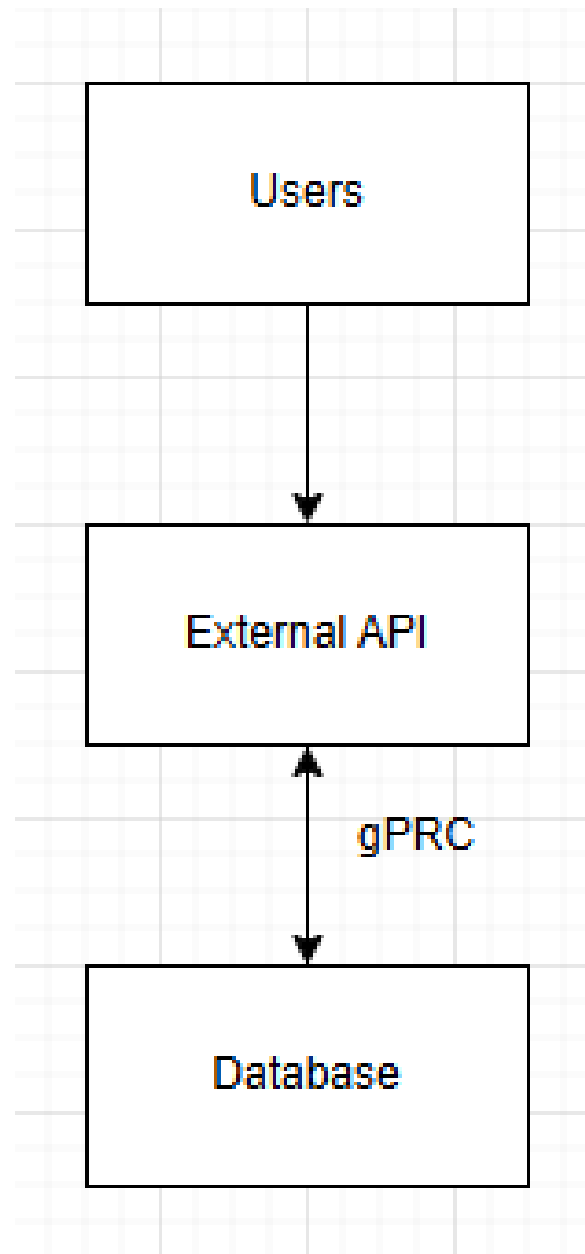
Liz @ Asset Cool

Tasks Attempted

- Task 1
- Option B

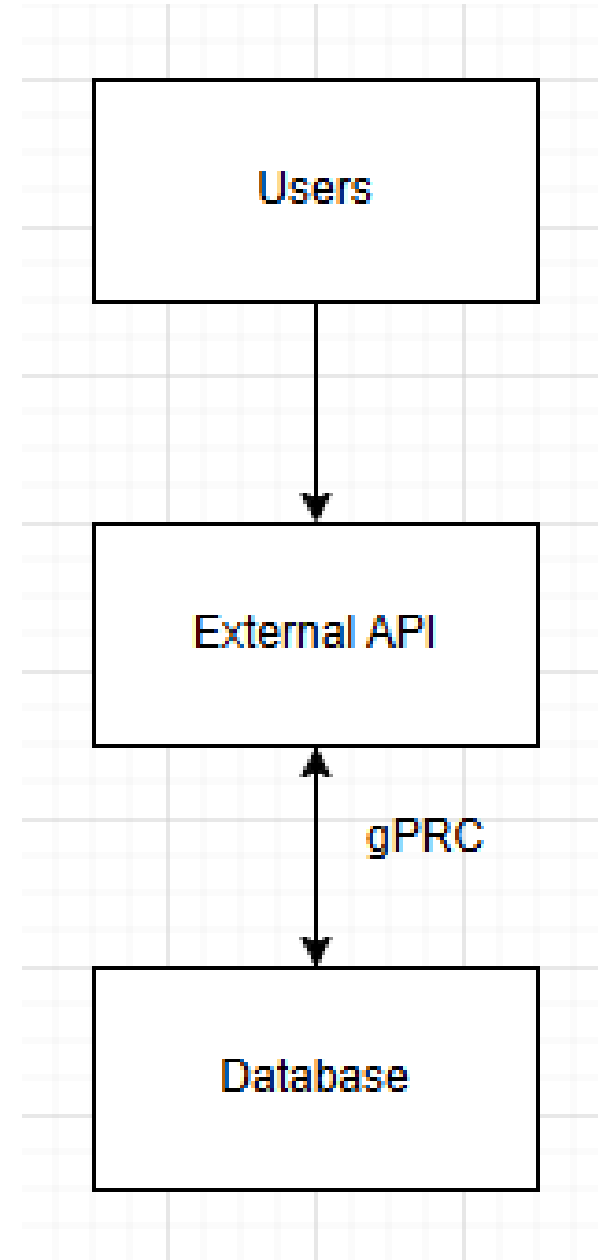
Task 1

Aim



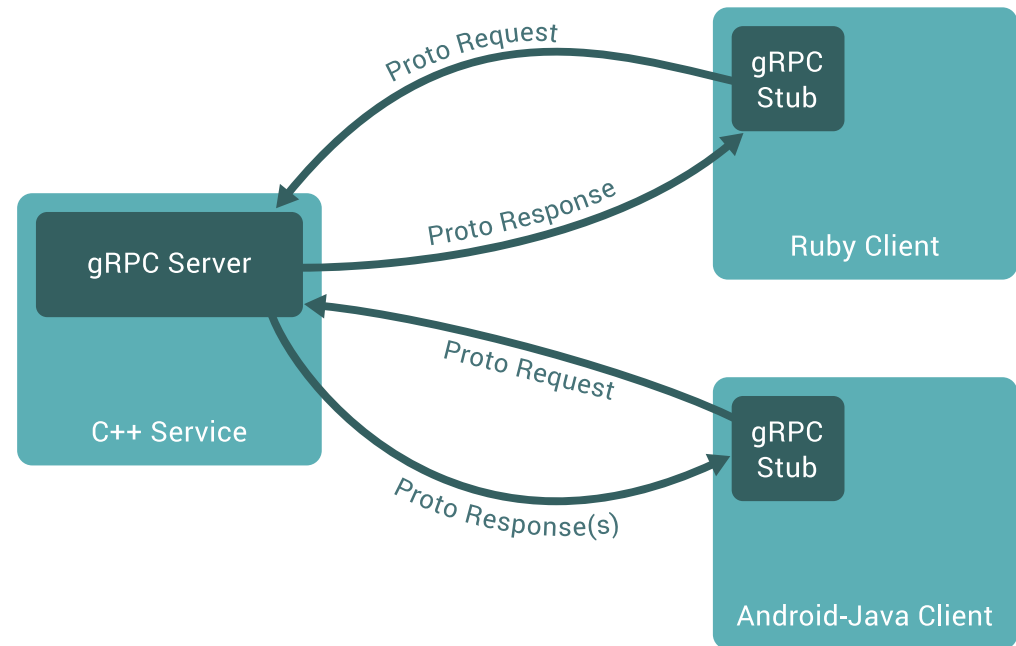
Aim

- Cloud-based system block architecture (using AWS or GCP) for access to an SQL database through an externally-facing gRPC API.

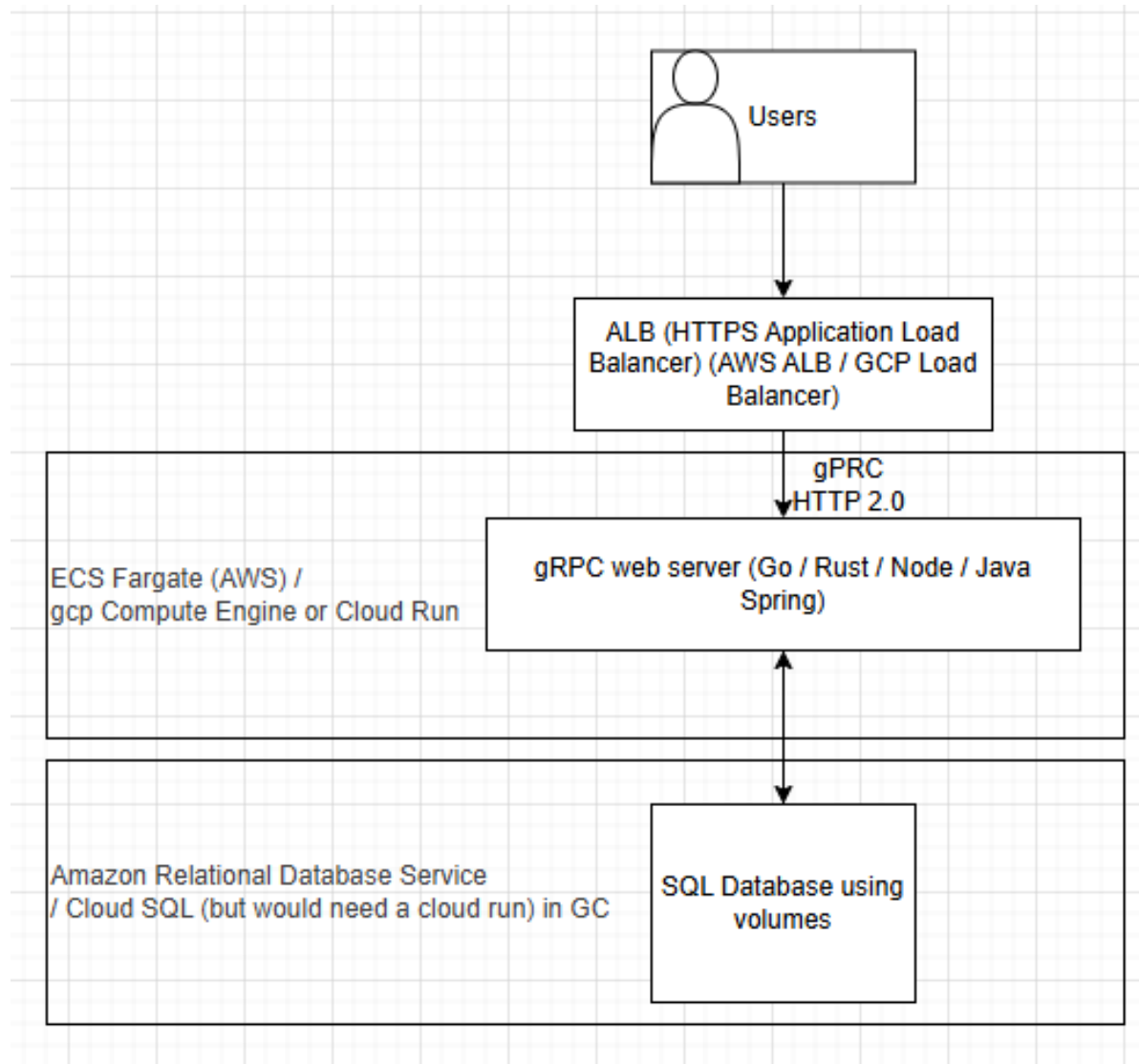


gRPC

- open-source framework
- high-performance, cross-platform, and language-agnostic Remote Procedure Calls (RPCs)
- Protocol Buffers to define service interfaces and messages, and HTTP/2 for transport
- bidirectional streaming, strong typing, and efficient binary serialization.
- <https://grpc.io/docs/what-is-grpc/introduction/>

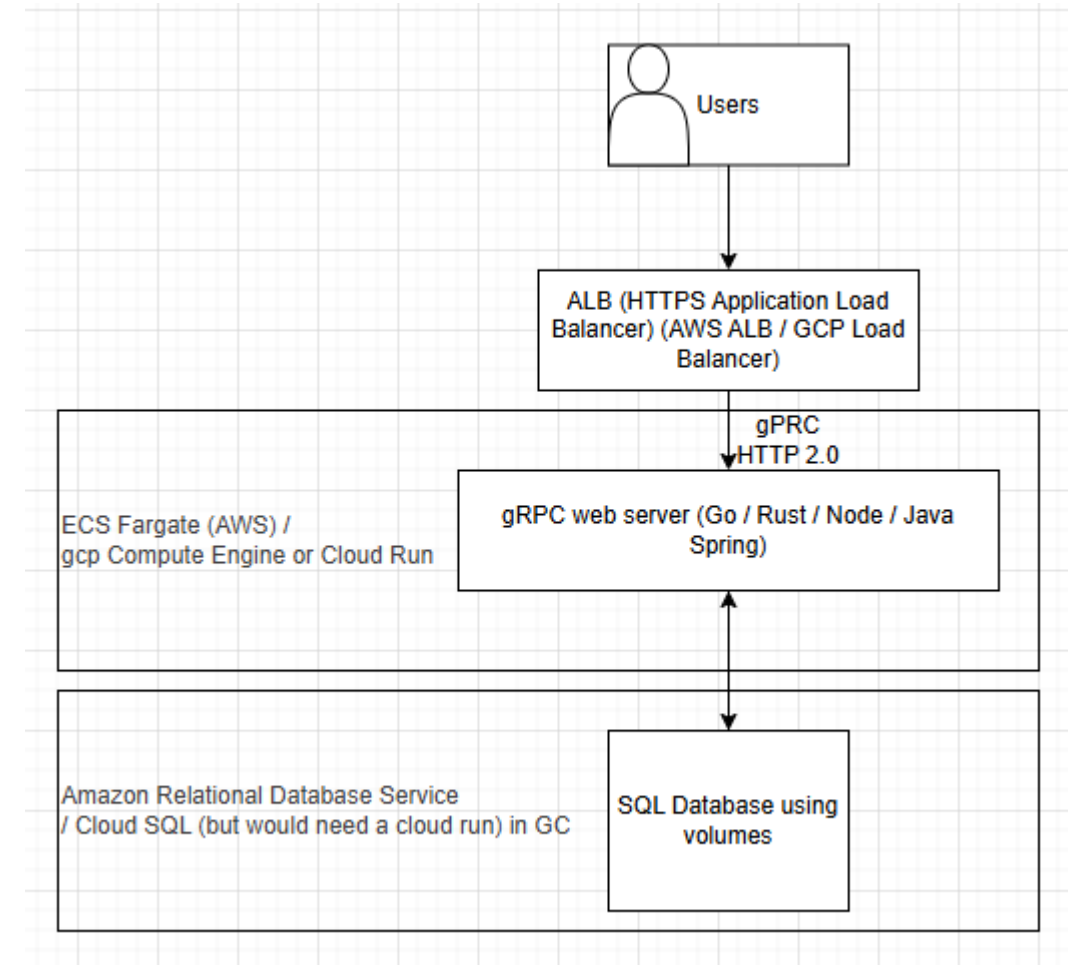


Simple Result



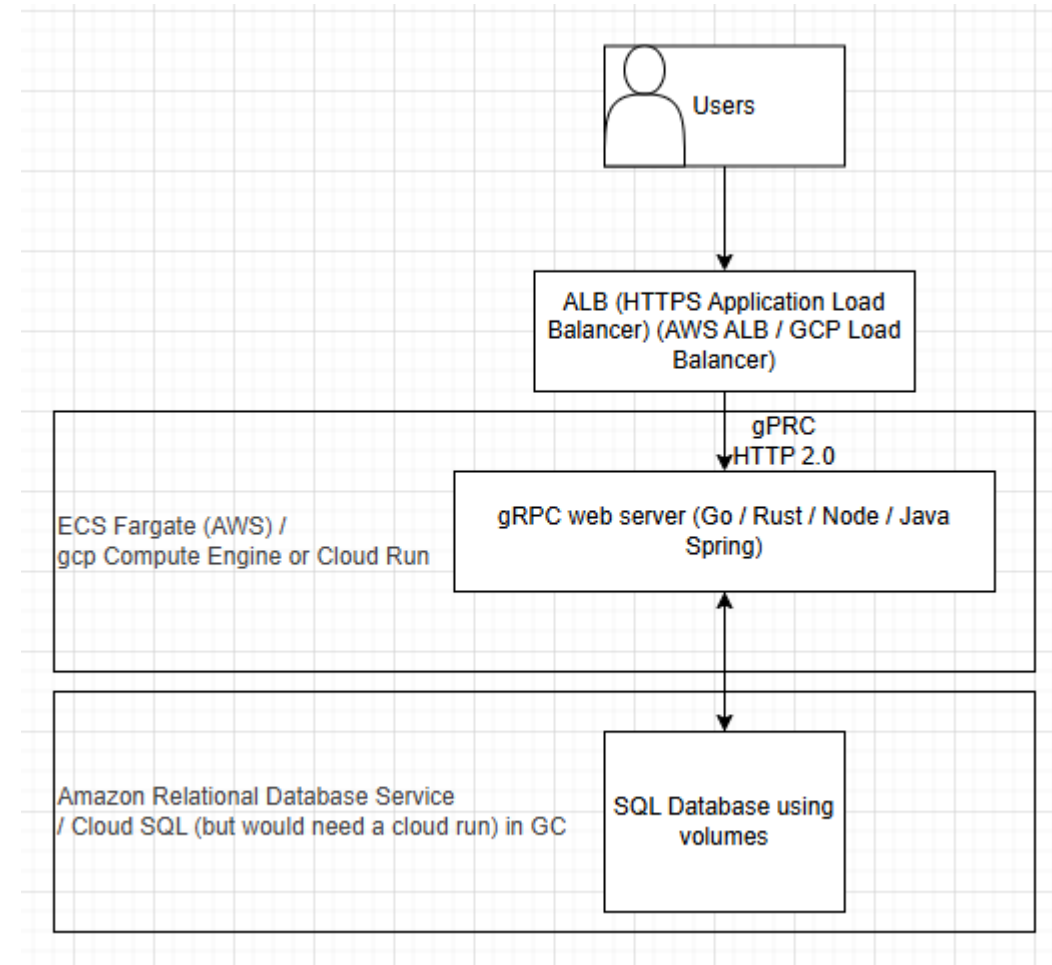
Load Balancer (ALB / GCP Load Balancer)

- A load balancer distributes incoming network traffic across a group of backend servers
- Add security
- Requirements: HTTP/2 and TLS encryption
- **Load balancer has a public IP address accessible from Internet**
- Everything behind the load balancer is private (containers, database)
- GCP – cheaper and simpler
- Certificates are needed
- <https://www.thredd.com/resources/building-thredd/implementing-grpc-on-aws>



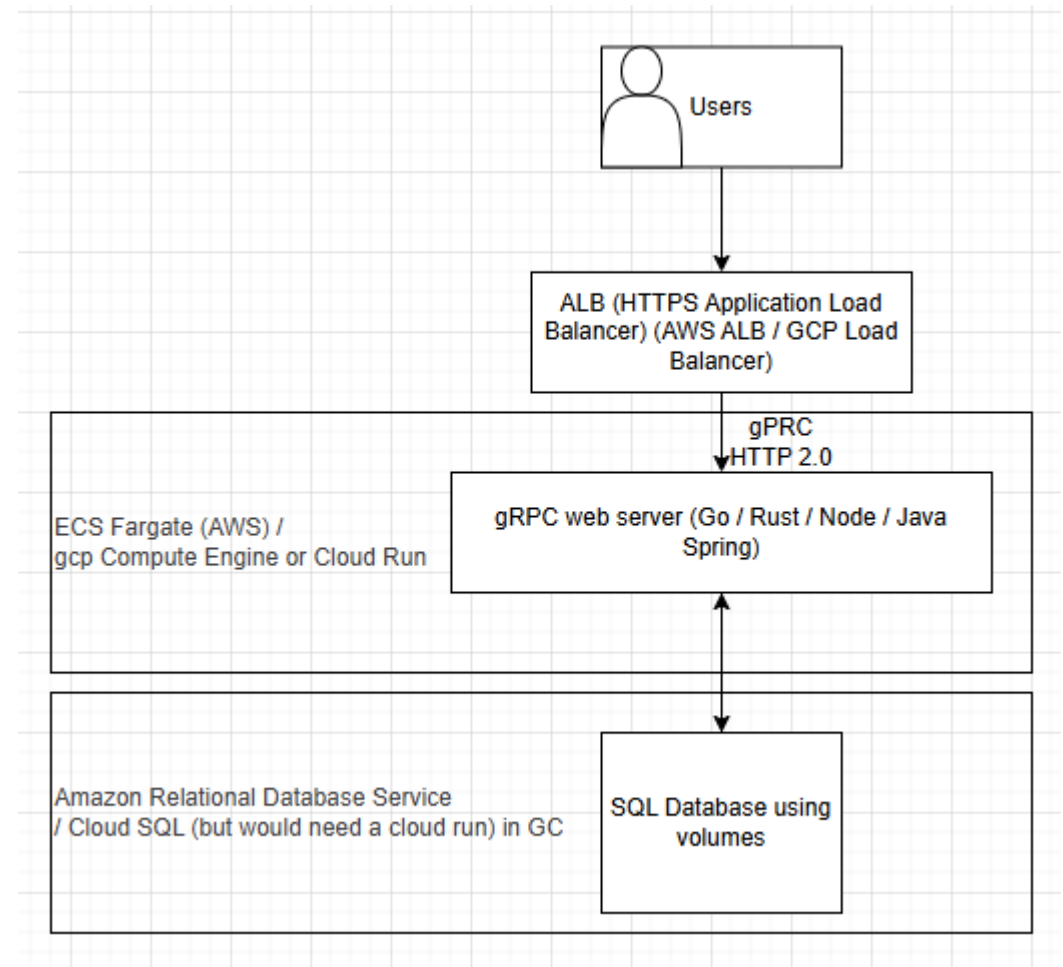
Compute Platform (EC2 / Cloud Run)

- Why - **Listen** for incoming gRPC requests, process the request and then shuttle to and from the database.
- Go/Rust/Java code as a gRPC web server lives here.
- Cloud Run – automatic server management, scaling, containerisation, LB integration, Cheap but limited config options (<https://medium.com/@o.hanhaliuk/google-cloud-run-vs-aws-ecs-fargate-2bcc49f0dd46>)
- Fargate –Serverless, auto scaling, built in containers, integrated LB, more complex to set up. AWS instructions on how to set it up <https://aws.amazon.com/blogs/opensource/containerize-and-deploy-a-grpc-application-on-aws-fargate/>
- Other Options: EKS / ECS on EC2 etc
 - Sum up – Differ in terms of control/flexibility/complexity
- EXAMPLE with fargate https://github.com/cheeyeo/grpc_aws_alb_example



Database

- RDS - Easier to set up, operate, and scale a relational database
- Persistent Storage of data
- Enterprise applications
- Cloud SQL very similar
- <https://www.economize.cloud/blog/aws-rds-vs-cloud-sql/>
- Other options: Aurora-managed.. Or DynamoDb or ElastiCache for real time ... Databricks
- <https://docs.aws.amazon.com/databases-on-aws-how-to-choose/>



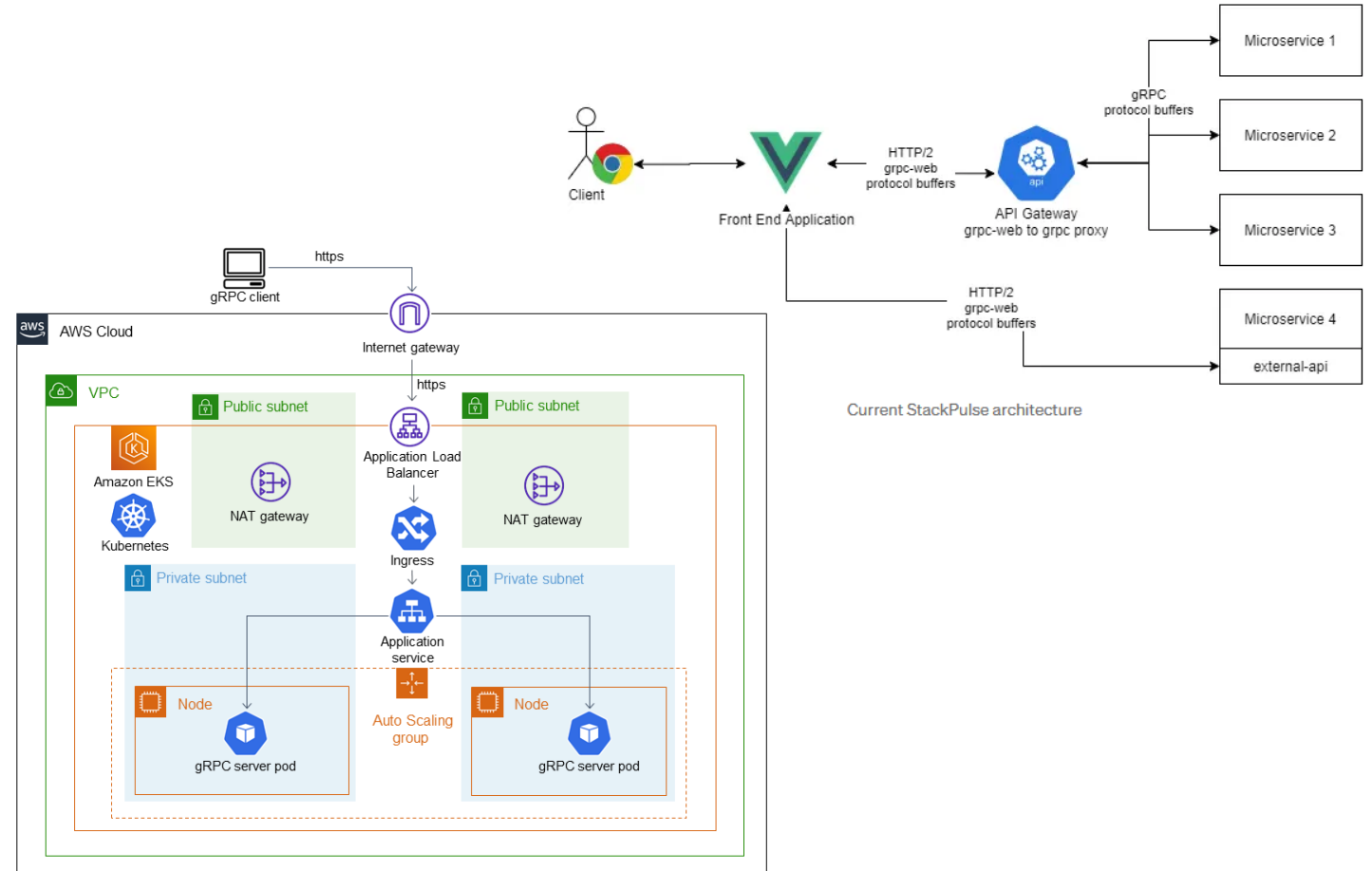
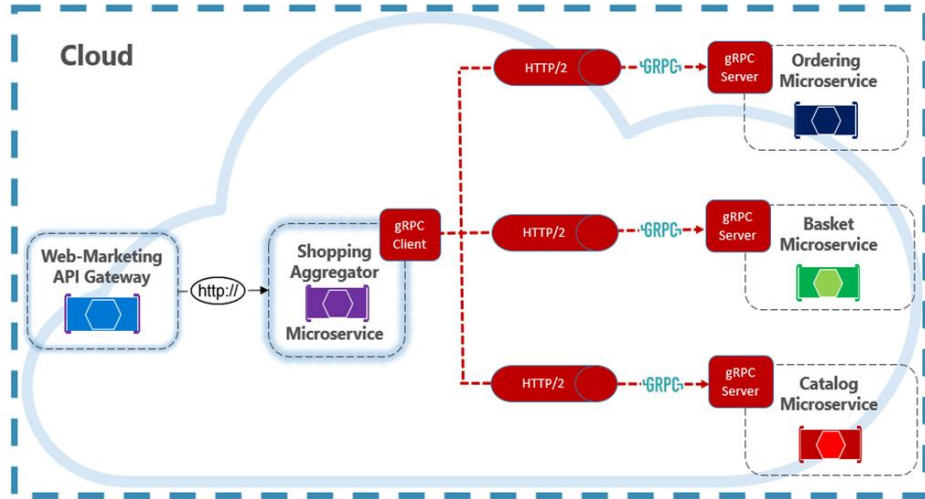
Add-ons : Need more research

- Firewalls
- Scaling
- Secrets / key vaults
- Monitoring
- Cache layers
- Subnets
- Database schemas

Interesting TidBits

- Databricks Connect communicates with the Databricks Clusters via gRPC over HTTP/2. - <https://docs.databricks.com/aws/en/dev-tools/databricks-connect/advanced>

Lots of mentions of microservices

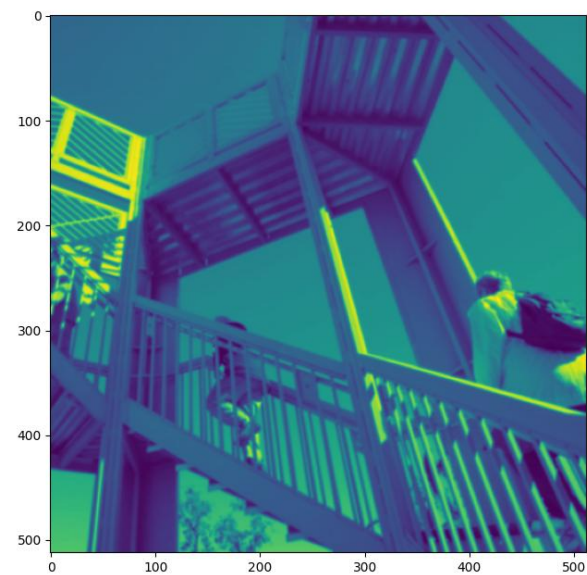
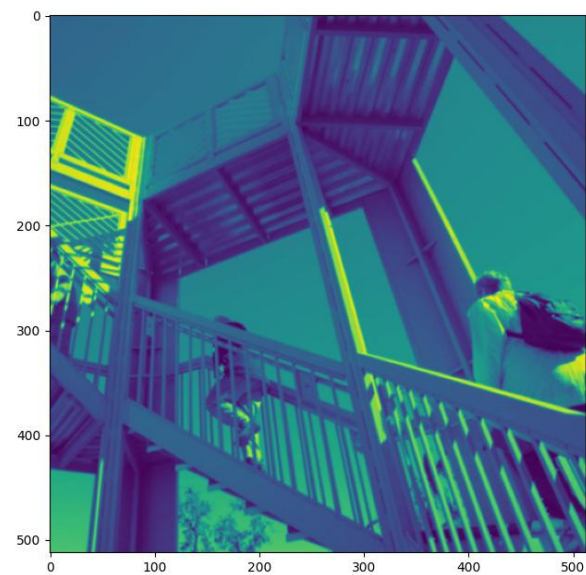
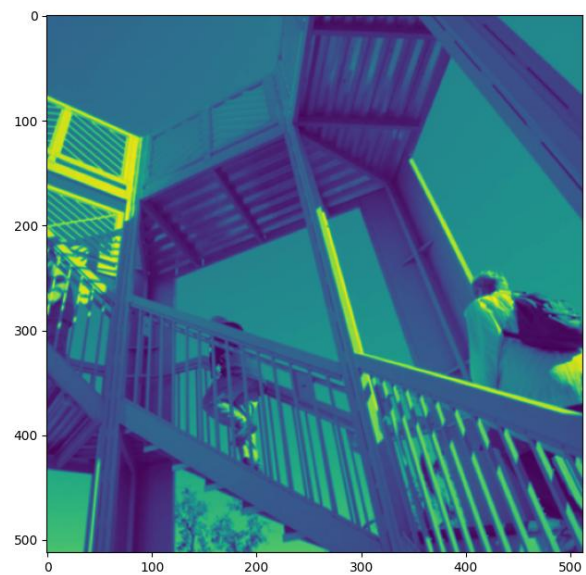
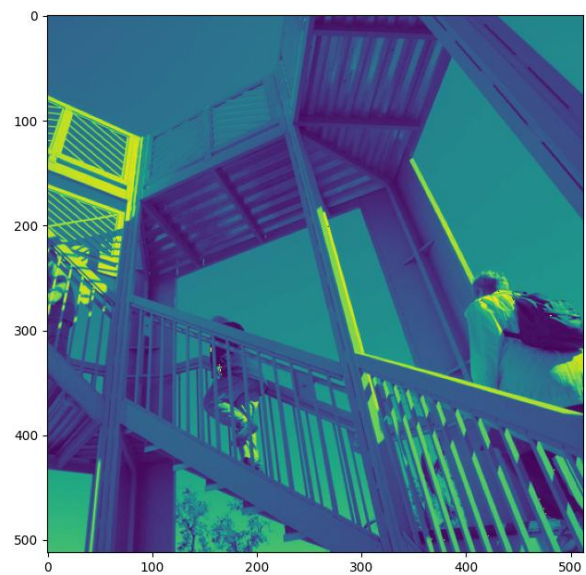


<https://www.google.com/url?sa=i&url=https%3A%2F%2Fmedium.com%2Fjavarevisited%2Fmicroservices-communication-using-grpc-protocol-dc3a2f8b648d&psig=AOvVaw0jsMGVsqWANrxP23cM6axm&ust=1760043170330000&source=images&cd=vfe&opi=89>

[Deploy a gRPC-based application on an Amazon EKS cluster and access it with an Application Load Balancer - AWS Prescriptive Guidance](#)

This uses a proxy because javascript doesn't support natively
https://www.google.com/url?sa=i&url=https%3A%2F%2Fkostyay.com%2Fgrpc-web-using-grpc-in-your-front-end-application-100178ea96e3&psig=AOvVaw0jsMGVsqWANrxP23cM6axm&ust=1760043170330000&source=images&cd=vfe&opi=89978449&ved=0CBkQjhxqFwoTCMCw7_C9IZADFQAAAAAAdAAAAABBN

Option B – Gaussian Blur



First Thoughts

- 3 x 3 matrix

1/16

1	2	1
2	4	2
1	2	1

1/273

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

1/1003

0	0	1	2	1	0	0
0	3	13	22	13	3	0
1	13	59	97	59	13	1
2	22	97	159	97	22	2
1	13	59	97	59	13	1
0	3	13	22	13	3	0
0	0	1	2	1	0	0

Code

```
def gaussian_blur_kernel(image_array, gaussian_matrix, stride):
    ...

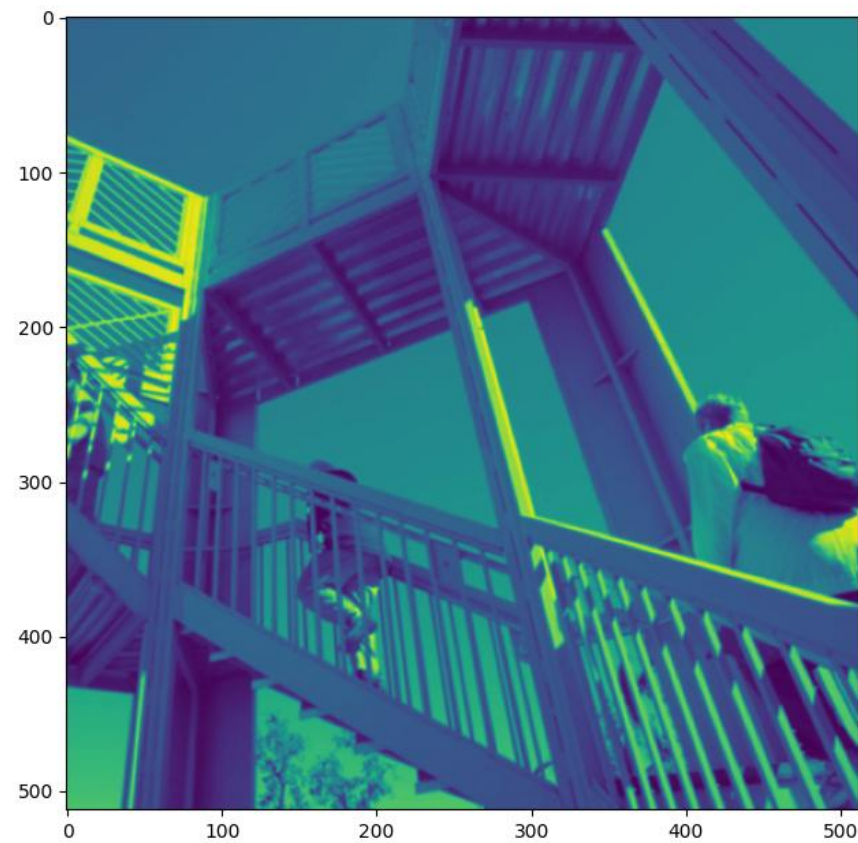
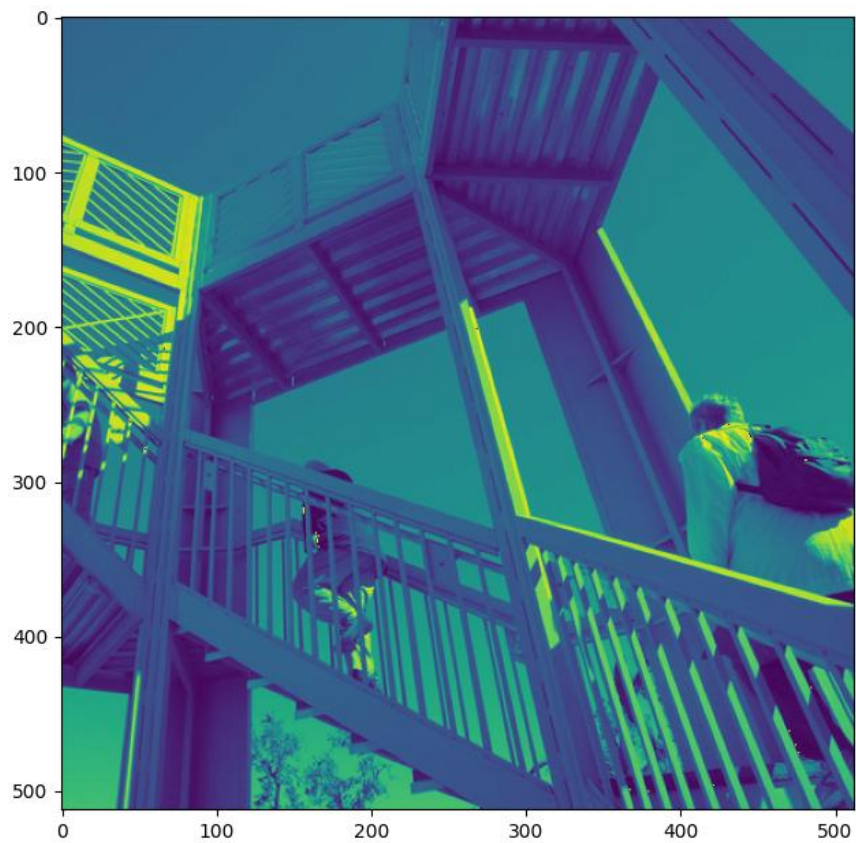
    Apply Gaussian filter: Using simple 3x3 normalised gaussian kernel
    ...

    # Create blank array to store the blurred image
    blurred_image = np.zeros(image_array.shape)

    # Add padding to the image
    padded_image = np.pad(image_array, pad_width=1, mode="edge")

    # Loop over every pixel in the image and apply the gaussian matrix
    for row_idx, row in enumerate(image_array):
        for pixel_idx in range(0, len(row), stride):
            # Add +1 offset to access the correct part of the padded image
            padded_row = row_idx + 1
            padded_col = pixel_idx + 1
            # Apply the gaussian matrix to the pixel and its surrounding neighbours
            blurred_image[row_idx][pixel_idx] = (
                padded_image[padded_row - 1][padded_col - 1] * gaussian_matrix[0][0]
                + padded_image[padded_row - 1][padded_col] * gaussian_matrix[0][1]
                + padded_image[padded_row - 1][padded_col + 1] * gaussian_matrix[0][2]
                + padded_image[padded_row][padded_col - 1] * gaussian_matrix[1][0]
                + padded_image[padded_row][padded_col] * gaussian_matrix[1][1] # Center
                + padded_image[padded_row][padded_col + 1] * gaussian_matrix[1][2]
                + padded_image[padded_row + 1][padded_col - 1] * gaussian_matrix[2][0]
                + padded_image[padded_row + 1][padded_col] * gaussian_matrix[2][1]
                + padded_image[padded_row + 1][padded_col + 1] * gaussian_matrix[2][2]
            ) // GAUSSIAN_MATRIX_WEIGHT # Normalise the image to ensure pixel value is 0-255
    return blurred_image
```

Result



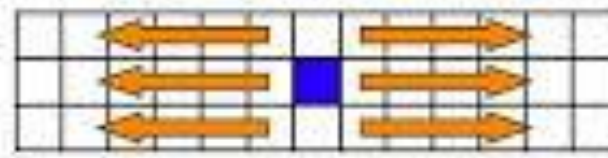
Tricky bits

- Making sure we add padding so that all the pixels can be blurred
 - Padding of all 0s?
 - Padding of closest neighbour?
- Size of filter, 3x3 is quite small. We could've used a larger filter to have more blur
- Normalizing the pixels (dividing by 16)
 - Grayscale pixels have a value between 0 and 255
 - If we keep multiplying and adding our values will all end up a lot darker.

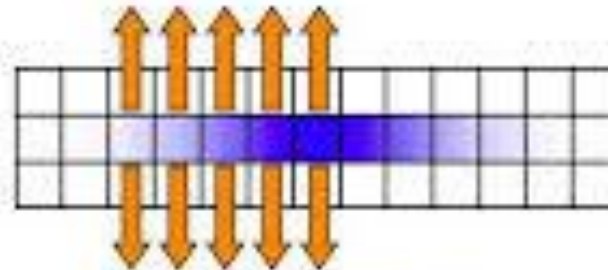
Are there any structural properties of the Gaussian kernel and convolution that you can exploit in order to improve the big-O complexity in the spatial domain?

Attempt 2

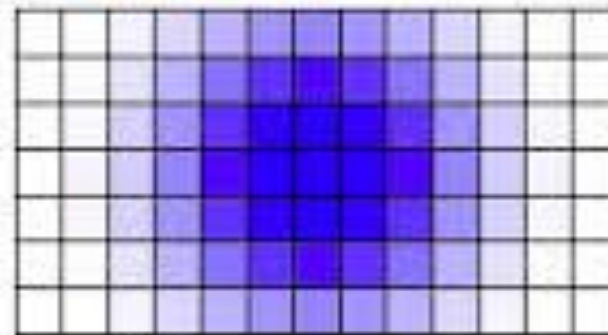
Better Big O == Reduce the number of loops



Blur the source
horizontally



Blur the blur
vertically



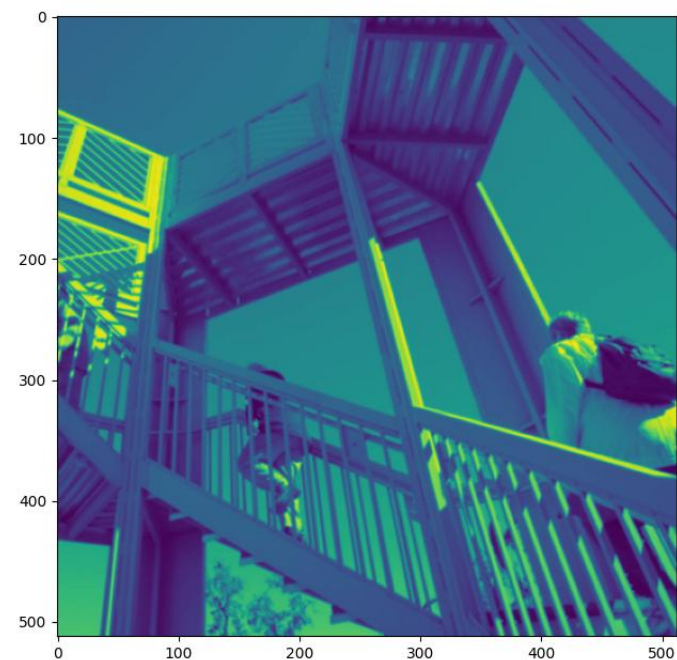
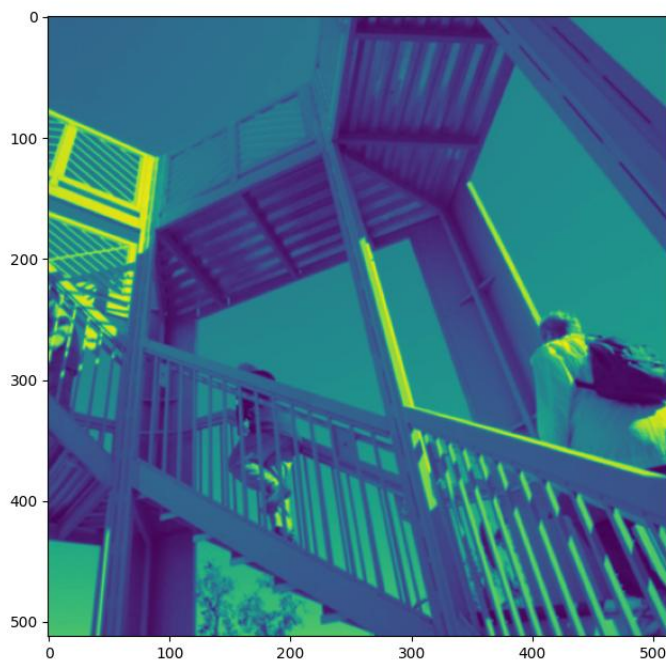
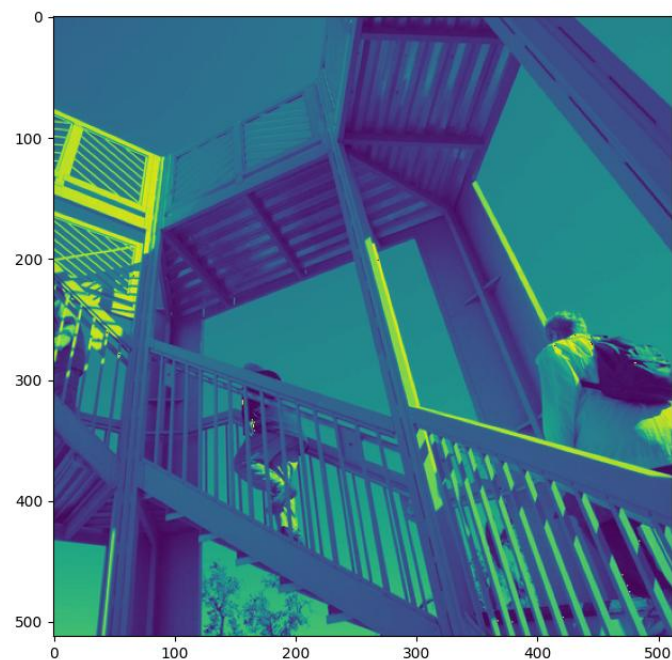
Result

Image taken from ATI's presentation

Code

```
57 def gaussian_blur_1d(image_array, axis, stride):
58     """
59     Use horizontal and vertical passes of 1D Gaussian filter
60     """
61
62     # Create blank array to store the blurred image
63     blurred_image = np.zeros(image_array.shape)
64
65     # Add padding to the image
66     padded_image = np.pad(image_array, pad_width=1, mode="edge")
67
68     # 1D Gaussian kernel
69     gaussian_1d = np.array([1, 2, 1])
70     gaussian_1d_weight = 4
71
72     if axis == 0: # Vertical blur, iterates over rows
73         for row_idx, row in enumerate(image_array):
74             for pixel_idx in range(0, len(row), stride):
75                 # Add +1 offset to access the correct part of the padded image
76                 padded_row = row_idx + 1
77                 padded_col = pixel_idx + 1
78                 # Apply the gaussian matrix to the pixel and its surrounding neighbours
79                 blurred_image[row_idx][pixel_idx] = (
80                     padded_image[padded_row - 1][padded_col] * gaussian_1d[0]
81                     + padded_image[padded_row][padded_col]
82                     * gaussian_1d[1] # Center pixel
83                     + padded_image[padded_row + 1][padded_col] * gaussian_1d[2]
84                 ) // gaussian_1d_weight # Normalise the image to ensure pixel value is 0-255
85
86     elif axis == 1: # Horizontal blur, iterates over columns
87         for row_idx, row in enumerate(image_array):
88             for pixel_idx in range(0, len(row), stride):
89                 # Add +1 offset to access the correct part of the padded image
90                 padded_row = row_idx + 1
91                 padded_col = pixel_idx + 1
92                 # Apply the gaussian matrix to the pixel and its surrounding neighbours
93                 blurred_image[row_idx][pixel_idx] = (
94                     padded_image[padded_row][padded_col - 1] * gaussian_1d[0]
95                     + padded_image[padded_row][padded_col]
96                     * gaussian_1d[1] # Center pixel
97                     + padded_image[padded_row][padded_col + 1] * gaussian_1d[2]
98                 ) // gaussian_1d_weight # Normalise the image to ensure pixel value is 0-255
99     return blurred_image
```


Result - Same



Tricky bits - Why is the Big O better second time round?

- In a matrix, for each pixel, we take the center and then do
 - 9 multiplications and (add 8 things together) = **17 operations per pixel**
 - The bigger the kernel, the more operations needed
 - Use the kernel size as $N \rightarrow 3 \times 3$ grid, $n = 3$, $n^2 = 9 \Rightarrow O(n^2)$
- Using vertical / horizontal
 - 3 multiplications (and 2 additions) and 3 multiplications (and 2 additions) = **10 operations per pixel**
 - Use the kernel size as $N \rightarrow 1 \times 3$ grid but now doing 2 passes.
 - Therefore, $2N \Rightarrow O(n)$